Experiment No. 01

Aim: To perform version control operations using Git.

LO Mapped: LO2

LO: Use Git effectively for version control, source code management, and collaborative team development.

Theory:

1. Introduction to Version Control Systems (VCS)

A Version Control System (VCS) acts as a historical record for a codebase, tracking every modification made to source files. By enabling developers to review past iterations, revert changes, and collaborate seamlessly, VCS serves as the essential foundation for Agile workflows and modern DevOps automation.

This format is ideal for reports or technical documentation where you want to explain the architectural differences clearly.

Centralized Version Control Systems (CVCS) operate on a client-server model, where a single central repository holds all versioned data. While this creates a unified source of truth, it introduces a significant single point of failure: if the central server goes offline, collaboration ceases, and hardware failure poses a risk to the entire project's history.

In contrast, Distributed Version Control Systems (DVCS), such as Git, eliminate this dependency by providing every developer with a full local copy of the project, including its entire historical log. This architecture ensures high redundancy and allows for offline productivity, as developers can commit changes and view history locally before syncing with others via remote architectures.

2. Git Architecture and Code Lifecycles

Git manages code through a structured four-stage workflow that moves files from local edits to global synchronization:

1. **Working Directory:** Your active workspace where you create and edit files. These changes are initially local and uncommitted.
2. **Staging Area (Index):** A preparation zone where you selectively "stage" changes. This allows you to curate exactly which modifications belong in your next commit, ensuring a clean and logical project history.
3. **Local Repository (.git):** The internal Git database located within your project. It stores your commit history, branch structure, and snapshots of your project, keeping them secure and versioned locally.
4. **Remote Repository:** An external server (like GitHub or GitLab) that serves as the hub for collaboration. It enables teams to share their local work, synchronize code, and maintain a unified project state.

3. Core Git Operations and Workflows

To successfully run a version-controlled project, Git operations are split into logical phases reflecting an automated engineering pipeline.

Phase A: Configuration & Initialization

Before you can effectively track a project, you must configure your identity and initialize your environment. This initial setup ensures a transparent history and enables Git's tracking capabilities.

- **Configure Identity (git config):** Set your name and email address. This tags every commit with your identity, providing a clear audit trail for collaborative history.
- **Initialize the Project (git init):** Transform a standard directory into a Git repository by creating a hidden. git folder. This acts as the engine that records all future changes.
- **Monitor Progress (git status):** Regularly check the health of your project. This command reveals which files are modified, staged, or untracked, giving you visibility into the current state of your working directory.

Phase B: The Local Tracking Loop

Before finalizing a commit, you must curate your changes. This loop allows you to selectively build your next snapshot without affecting your actual code files.

- git add (Staging): Moves specific file modifications from your working directory into the staging index. This "prepares" the files, marking them as ready to be part of the next commit.
- git rm --cached / git restore --staged (Unstaging): Removes files from the staging index if you decide they aren't ready to be committed. Crucially, this only updates the Git index; your actual file content remains untouched.
- git mv (Relocating): Renames or moves files within the project. This command handles the file system change while simultaneously registering the new path in Git, keeping your history consistent.

Phase C: Inspection & Delta Auditing

Visibility is critical in Agile sprints to ensure changes match task descriptions.

- **Log Diagnostics (git log, git log --oneline):** Displays sequential arrays of chronological commits, capturing cryptographic SHA hashes, author footprints, and contextual messages.
- **Difference Analytics (git diff, git diff --staged):** Compares code variations line by line, demonstrating additions and deletions before or after staging them.

Phase D: Agile Branching & Workspace Isolation

Agile workflows isolate user stories and bug fixes into dedicated sandboxes before integrating them into production lines.

- **Branch Allocation (git branch, git branch -d):** Creates or removes independent structural pointers within the commit history graph.
- **Context Switching (git checkout):** Transitions the active working directory space from one tracking branch line to another.
- **Integration (git merge):** Reconciles and combines tracking logs from a specialized feature line directly into a target base branch (e.g., main).

Phase E: Stash Index Manipulation

- **Data Shelving (git stash, git stash pop):** Temporarily saves uncommitted, incomplete feature states onto an internal memory stack. This cleans the working directory instantly, allowing developers to switch tasks and later re-apply their cached modifications.

4. Deep-Dive: The 7 Mandated DevOps Operations

The following seven core operations dictate how code changes are validated, stored, secured, shared, and recovered across team-driven software environments.

[Local Workspace] ----(Commit)----> [Local Repository] ----(Push)----> [Remote Server]

[Local Workspace] <---(Clone/Pull)---------- [Remote Server]

1. Commit

Definition: A commit serves as an immutable cryptographic snapshot that preserves the exact state of your staged changes at a specific moment in the project's lifecycle.

Mechanism: Every commit is assigned a unique identifier (SHA-1 or SHA-256 hash) that functions as a digital fingerprint. This hash encapsulates the complete project hierarchy, the lineage of previous commits, author metadata, and the descriptive message. Within Agile workflows, the commit acts as a critical baseline, confirming that a discrete unit of work is validated, documented, and ready for integration.

2. Authorisation

Purpose: Serves as the security validation layer, verifying that a developer has the necessary permissions to read from or write to a remote repository.

Mechanism: Git secures these interactions using HTTPS access tokens or cryptographic SSH keys. By integrating with tools like the Git Credential Manager, the system validates compliance before authorizing any code transfer. This process ensures that all interactions are authenticated, creating a reliable audit trail for secure DevOps deployment pipelines.

3. Push

Definition: The operational process of uploading local repository commits to a remote server.

Mechanism: Pushing synchronizes your local branch history with the remote host. This updates the shared codebase, grants the team visibility into your progress, and typically triggers automated CI/CD pipelines.

4. Pull

Definition: A git pull is an inbound operation that retrieves updates from a remote repository and automatically merges them into your active local branch.

Mechanism: It is essentially a "two-in-one" command. When you run git pull, Git performs two distinct actions in rapid succession:

git fetch: Downloads the latest commit history and data from the remote source.

git merge: Immediately integrates those downloaded changes into your current working directory.

5. Clone

Cloning serves as the primary method for initializing a local project environment by performing a complete, deep copy of an existing remote repository. When you execute a clone, Git automatically performs four critical tasks: it sets up a local directory, downloads the entire project history and tracking database, configures a remote shortcut (typically named origin), and performs a checkout of the default primary branch to prepare your workspace for immediate development.

6. Revert

Definition: A safe, non-destructive operation that reverses the impact of a previous commit without altering the project's historical timeline.

Mechanism: git revert functions by identifying a specific commit, calculating its inverse, and applying those changes as a new, forward-moving commit. Because it leaves the existing history intact, it is considered the safest approach for rectifying errors in shared or production environments.

7. Rollback (Reset Mechanics)

A rollback is a structural adjustment that shifts your branch pointer back to a previous commit, effectively discarding any subsequent work from the project timeline. Depending on the level of preservation required, Git offers three distinct reset protocols:

- Soft Reset: Reverts the branch history but keeps your modified files "staged" and ready to be committed again. Use this if you want to bundle changes differently.
- Mixed Reset (Default): Reverts the branch history and clears the staging area. Your changes remain in your local directory as "unstaged" modifications, giving you a clean slate to re-organize.
- Hard Reset: A destructive operation that aligns your entire workspace with the target commit. It discards all staged and unstaged changes, permanently wiping away any work made after the target point.

Implementation:

git config

git init

```
C:\Users\yogini bhatia>git config --global user.name "Yogini Bha
tia"

C:\Users\yogini bhatia>git config --global user.email "your-emai
l@example.com"

C:\Users\yogini bhatia>cd "C:\Users\yogini bhatia\skillnova-pro"


C:\Users\yogini bhatia\skillnova-pro>git init
Reinitialized existing Git repository in C:/Users/yogini bhatia/
skillnova-pro/.git/

C:\Users\yogini bhatia\skillnova-pro>
```

git clone

```
C:\Users\yogini bhatia>git clone https://github.com/Yogini21-jpg/skillnova-pro
Cloning into 'skillnova-pro'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 0), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), done.

C:\Users\yogini bhatia>cd skillnova-pro
```

git add .
git commit
git pull
git checkout main

```
C:\Users\yogini bhatia\skillnova-pro>git status
On branch feature/my-new-update
nothing to commit, working tree clean

C:\Users\yogini bhatia\skillnova-pro>git add .

C:\Users\yogini bhatia\skillnova-pro>git commit -m "Add feature: update user profile logic"
On branch feature/my-new-update
nothing to commit, working tree clean

C:\Users\yogini bhatia\skillnova-pro>git push -u origin feature/my-new-update
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/my-new-update' on GitHub by visiting:
remote:      https://github.com/Yogini21-jpg/skillnova-pro/pull/new/feature/my-new-update
remote:
To https://github.com/Yogini21-jpg/skillnova-pro
 * [new branch]      feature/my-new-update -> feature/my-new-update
branch 'feature/my-new-update' set up to track 'origin/feature/my-new-update'.

C:\Users\yogini bhatia\skillnova-pro>git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

git merge
git push
git checkout

```
C:\Users\yogini bhatia\skillnova-pro>git pull origin main
From https://github.com/Yogini21-jpg/skillnova-pro
 * branch            main       -> FETCH_HEAD
Already up to date.

C:\Users\yogini bhatia\skillnova-pro>git merge feature/my-new-update
Already up to date.

C:\Users\yogini bhatia\skillnova-pro>git push origin main
Everything up-to-date

C:\Users\yogini bhatia\skillnova-pro>git branch -d feature/my-new-update
Deleted branch feature/my-new-update (was e98a46c).

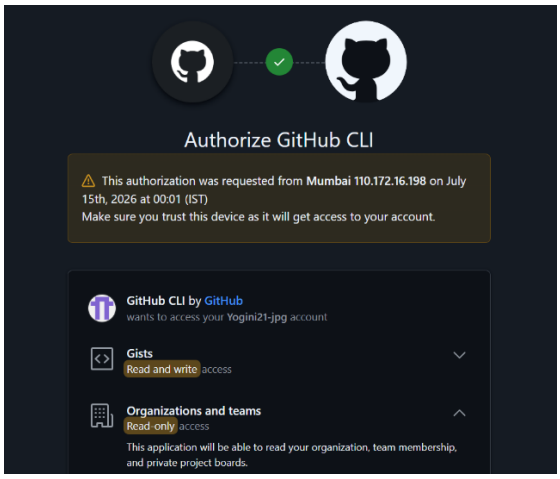
C:\Users\yogini bhatia\skillnova-pro>git checkout -b feature/your-next-feature-name
Switched to a new branch 'feature/your-next-feature-name'
```

Collaborating authorization with another user

```
C:\Users\yogini bhatia\skillnova-pro>gh api --method PUT /repos/Yogini21-jpg/skillnova-pro/collaborators/GYAAN-ACHANTANI -f permissio
n=write
{
  "id": 325696016,
  "node_id": "RI_kwDORfcyuM4TaboQ",
  "repository": {
    "id": 1173828280,
    "node_id": "R_kgDORfcyuA",
    "name": "skillnova-pro",
    "full_name": "Yogini21-jpg/skillnova-pro",
    "private": false,
    "owner": {
      "login": "Yogini21-jpg",
      "id": 222478425,
      "node_id": "U_kgDODULAWQ",
      "avatar_url": "https://avatars.githubusercontent.com/u/222478425?v=4",
      "gravatar_id": "",
      "url": "https://api.github.com/users/Yogini21-jpg",
      "html_url": "https://github.com/Yogini21-jpg",
      "followers_url": "https://api.github.com/users/Yogini21-jpg/followers",
      "following_url": "https://api.github.com/users/Yogini21-jpg/following{/other_user}",
      "gists_url": "https://api.github.com/users/Yogini21-jpg/gists{/gist_id}",
      "starred_url": "https://api.github.com/users/Yogini21-jpg/starred{/owner}{/repo}",
      "subscriptions_url": "https://api.github.com/users/Yogini21-jpg/subscriptions",
      "organizations_url": "https://api.github.com/users/Yogini21-jpg/orgs",
      "repos_url": "https://api.github.com/users/Yogini21-jpg/repos",
      "events_url": "https://api.github.com/users/Yogini21-jpg/events{/privacy}",
      "received_events_url": "https://api.github.com/users/Yogini21-jpg/received_events",
      "type": "User",
```

```
  },
  "invitee": {
    "login": "GYAAN-ACHANTANI",
    "id": 216299081,
    "node_id": "U_kgDODOR2SQ",
    "avatar_url": "https://avatars.githubusercontent.com/u/216299081?v=4",
    "gravatar_id": "",
    "url": "https://api.github.com/users/GYAAN-ACHANTANI",
    "html_url": "https://github.com/GYAAN-ACHANTANI",
    "followers_url": "https://api.github.com/users/GYAAN-ACHANTANI/followers",
    "following_url": "https://api.github.com/users/GYAAN-ACHANTANI/following{/other_user}",
    "gists_url": "https://api.github.com/users/GYAAN-ACHANTANI/gists{/gist_id}",
    "starred_url": "https://api.github.com/users/GYAAN-ACHANTANI/starred{/owner}{/repo}",
    "subscriptions_url": "https://api.github.com/users/GYAAN-ACHANTANI/subscriptions",
    "organizations_url": "https://api.github.com/users/GYAAN-ACHANTANI/orgs",
    "repos_url": "https://api.github.com/users/GYAAN-ACHANTANI/repos",
    "events_url": "https://api.github.com/users/GYAAN-ACHANTANI/events{/privacy}",
    "received_events_url": "https://api.github.com/users/GYAAN-ACHANTANI/received_events",
    "type": "User",
    "user_view_type": "public",
    "site_admin": false
  },
  "inviter": {
    "login": "Yogini21-jpg",
    "id": 222478425,
    "node_id": "U_kgDODULAWQ",
    "avatar_url": "https://avatars.githubusercontent.com/u/222478425?v=4",
    "gravatar_id": "",
    "url": "https://api.github.com/users/Yogini21-jpg",
    "html_url": "https://github.com/Yogini21-jpg",
    "followers_url": "https://api.github.com/users/Yogini21-jpg/followers",
```

Authorization access



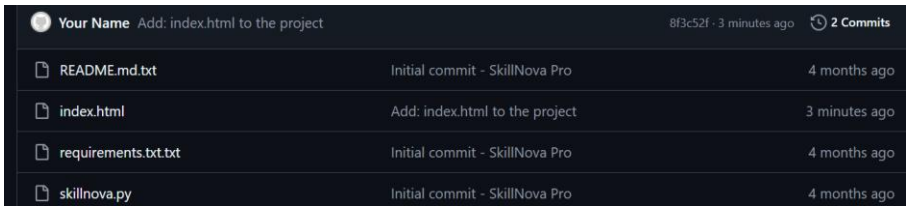
Deleting collaboration permissions or Reverting back access

```
C:\Users\yogini bhatia\skillnova-pro>gh api --method DELETE /repos/Yogini21-jpg/skillnova-pro/collaborators/GYAAN-ACHANTANI

C:\Users\yogini bhatia\skillnova-pro>gh api /repos/Yogini21-jpg/skillnova-pro/collaborators
[
  {
    "login": "Yogini21-jpg",
    "id": 222478425,
    "node_id": "U_kgDODULAWQ",
    "avatar_url": "https://avatars.githubusercontent.com/u/222478425?v=4",
    "gravatar_id": "",
    "url": "https://api.github.com/users/Yogini21-jpg",
    "html_url": "https://github.com/Yogini21-jpg",
    "followers_url": "https://api.github.com/users/Yogini21-jpg/followers",
    "following_url": "https://api.github.com/users/Yogini21-jpg/following{/other_user}",
    "gists_url": "https://api.github.com/users/Yogini21-jpg/gists{/gist_id}",
    "starred_url": "https://api.github.com/users/Yogini21-jpg/starred{/owner}{/repo}",
    "subscriptions_url": "https://api.github.com/users/Yogini21-jpg/subscriptions",
    "organizations_url": "https://api.github.com/users/Yogini21-jpg/orgs",
    "repos_url": "https://api.github.com/users/Yogini21-jpg/repos",
    "events_url": "https://api.github.com/users/Yogini21-jpg/events{/privacy}",
    "received_events_url": "https://api.github.com/users/Yogini21-jpg/received_events",
    "type": "User",
    "user_view_type": "public",
    "site_admin": false,
    "permissions": {
      "admin": true,
      "maintain": true,
      "push": true,
      "triage": true,
      "pull": true
    },
    "role_name": "admin"
```

git commit
Adding a file

```
C:\Users\yogini bhatia\skillnova-pro>git commit -m "Add: index.h
tml to the project"
[main 8f3c52f] Add: index.html to the project
 1 file changed, 25 insertions(+)
 create mode 100644 index.html
```



Checking if the file is added and checking its contents followed by committing the file

```
C:\Users\yogini bhatia\skillnova-pro>git log -p -1
commit 8f3c52f33dac97ae6dba182662fe716e0aae0a59 (HEAD -> main, origin/main, origin/HEAD)
Author: Your Name <your-email@example.com>
Date:   Wed Jul 15 00:19:07 2026 +0530

    Add: index.html to the project

diff --git a/index.html b/index.html
new file mode 100644
index 0000000..c1f69f9
--- /dev/null
+++ b/index.html
@@ -0,0 +1,25 @@
+<p>First paragraph. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore
gna
+aliqua. Arcu cursus vitae congue mauris. In nisl nisi scelerisque eu ultrices vitae auctor eu augue. Nisi est sit amet
+facilisis magna. Diam sit amet nisl suscipit adipiscing bibendum est ultricies integer. Quis ipsum suspendisse ultrices
+gravida dictum fusce ut. Euismod elementum nisi quis eleifend. Habitant morbi tristique senectus et. Amet nisl suscipit
+adipiscing bibendum est ultricies integer. Viverra orci sagittis eu volutpat odio facilisis mauris sit. Nisi quis
+eleifend quam adipiscing. Neque convallis a cras semper auctor neque vitae. Magna fermentum iaculis eu non. Vivamus arcu
+felis bibendum ut tristique et. Justo nec ultrices dui sapien eget mi. In vitae turpis massa sed elementum tempus. Eu
+facilisis sed odio morbi quis commodo. Sagittis aliquam malesuada bibendum arcu vitae elementum curabitur vitae.</p>
+
:...skipping...
commit 8f3c52f33dac97ae6dba182662fe716e0aae0a59 (HEAD -> main, origin/main, origin/HEAD)
Author: Your Name <your-email@example.com>
Date:   Wed Jul 15 00:19:07 2026 +0530

    Add: index.html to the project
```

Adding more changes to the file and pushing it to git

git checkout

```
C:\Users\yogini bhatia\skillnova-pro>git add style.css

C:\Users\yogini bhatia\skillnova-pro>git commit -m "Style: Add basic CSS layout to project"
[feature/setup-styling bbff7c2] Style: Add basic CSS layout to project
 1 file changed, 1 insertion(+)
 create mode 100644 style.css

C:\Users\yogini bhatia\skillnova-pro>git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

C:\Users\yogini bhatia\skillnova-pro>git merge feature/setup-styling
Updating 8f3c52f..bbff7c2
Fast-forward
 style.css | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 style.css

C:\Users\yogini bhatia\skillnova-pro>git branch -d feature/setup-styling
Deleted branch feature/setup-styling (was bbff7c2).

C:\Users\yogini bhatia\skillnova-pro>git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 378 bytes | 378.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Yogini21-jpg/skillnova-pro
   8f3c52f..bbff7c2  main -> main
```

Agile Task

Adding CSS changes directly

```
C:\Users\yogini bhatia\skillnova-pro>git checkout -b feature/setup-styling
Switched to a new branch 'feature/setup-styling'

C:\Users\yogini bhatia\skillnova-pro>echo "body { font-family: Arial, sans-serif; margin: 20px; background-color: #f4f4f4; }" > style
.css

C:\Users\yogini bhatia\skillnova-pro>git add style.css
```

git status

git restore

git stash

git stash list

git stash pop

```
(base) PS C:\Users\yogini bhatia\skillnova-pro> Add-Content -Path index.html -Value "<!-- Testing stash operation -->"
(base) PS C:\Users\yogini bhatia\skillnova-pro> git status
On branch feature/update-content
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
(base) PS C:\Users\yogini bhatia\skillnova-pro> git stash
warning: in the working copy of 'index.html', LF will be replaced by CRLF the next time Git touches it
Saved working directory and index state WIP on feature/update-content: bbff7c2 Style: Add basic CSS layout to project
(base) PS C:\Users\yogini bhatia\skillnova-pro> git stash list
stash@{0}: WIP on feature/update-content: bbff7c2 Style: Add basic CSS layout to project
(base) PS C:\Users\yogini bhatia\skillnova-pro> git stash pop
On branch feature/update-content
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (9216343d882c75bd792cf45598ced50d1b6472f7)
```

Conclusion:

In conclusion, this experiment successfully demonstrated the practical application of version control operations using the Git distributed framework. By establishing localized tracking loops, executing branch isolation workflows, and deploying error recovery mechanisms, the core competencies of source code management outlined in Learning Outcome 2 (LO2) were achieved. Furthermore, performing these operations on my own projects underscored the importance of configuration consistency and file tracking as the essential foundation for managing professional software development workflows.

LO2 mapped successfully.